# Activity 2 - (5%)

## Due Date - Sunday October 11, 2026 (Late Policy: 10% per day)

## Note: You can implement in any language of your choice

1. **Game Theory (60%)**

   Design a move generator for the game Connect Four (a 6 row by 7 column board) based on the Monte Carlo tree search algorithm in Figure 6.11 in the text.
   - **selection** this should be based on some utility function which incorporates both playout (stored from previous results) and objective (from rules of the game) parts
   - **expansion** at each level in the tree, select *x* nodes to expand
   - **simulation** when you get to the fixed level in the tree....simulate with a random playout policy
   - repeat a configurable number of times

   Experiment with your utility function, the number of cycles, expansion nodes and the height of the tree. Provide a report containing the data collected and analyse/reflect on the results. Does the simulator always generate the same move? What causes differences or similarities in your results?

   Design Setup:
   Agent plays through connect 4 on a 6x7 board grid.
   Iterations are based on board, player, width of the tree as expand_x ,max depth of tree, exploration parameter (c), and weight of the tree.

   Node select based on UCB1 equation, expand to child node, simulate playthrough to terminal conclusion, and backpropagates to detail best path to win the game.

   An experiment using Monte Carlo Tree Search ran through:
   - 100,500, and 2000 iterations
   - Expanding width of x by 2,4,6
   - Max depth of 2,4,7
   - Width of 0,0.5,1
   - Exploration Parameter(c) of 0.5,1.4,3

   Tests inputs of [column, player(1 or 2)]:
   (0,1),(3,2),(6,1),(4,2),(1,1),(5,2),(3,1)

   Data:
   Run#1

| parameter | value | move_counts | avg_seconds |
|---|---|---|---|
| iterations | 100 | {1: 5, 5: 6, 4: 4, 3: 3, 0: 1, 2: 1} | 0.08 |

| | | | |
|---|---|---|---|
| iterations | 500 | {0: 3, 3: 5, 2: 5, 4: 3, 5: 2, 1: 2} | 0.376 |
| iterations | 2000 | {1: 3, 6: 3, 3: 3, 4: 2, 0: 6, 5: 3} | 1.54 |
| expand_x | 2 | {5: 4, 2: 3, 4: 4, 3: 4, 6: 1, 0: 2, 1: 2} | 0.383 |
| expand_x | 4 | {6: 3, 1: 3, 5: 5, 4: 2, 2: 3, 0: 3, 3: 1} | 0.413 |
| expand_x | 7 | {4: 4, 0: 3, 2: 2, 5: 4, 1: 4, 3: 1, 6: 2} | 0.729 |
| max_depth | 2 | {6: 5, 5: 4, 1: 1, 2: 3, 4: 4, 3: 1, 0: 2} | 0.962 |
| max_depth | 4 | {6: 6, 5: 5, 4: 2, 3: 4, 1: 1, 0: 1, 2: 1} | 1.029 |
| max_depth | 6 | {1: 4, 5: 5, 6: 1, 0: 3, 3: 2, 4: 3, 2: 2} | 1.083 |
| w | 0 | {4: 2, 3: 4, 6: 3, 5: 6, 1: 3, 0: 1, 2: 1} | 1.051 |
| w | 0.5 | {3: 13, 0: 1, 2: 1, 5: 2, 6: 1, 1: 1, 4: 1} | 1.023 |
| w | 1 | {4: 1, 5: 3, 6: 2, 3: 10, 1: 1, 0: 3} | 0.976 |
| c | 0.5 | {1: 6, 3: 2, 2: 3, 6: 3, 0: 4, 5: 2} | 1.053 |
| c | 1.4 | {3: 2, 6: 5, 0: 4, 5: 3, 2: 3, 1: 3} | 1.103 |
| c | 3 | {0: 2, 6: 2, 4: 4, 5: 5, 3: 3, 2: 1, 1: 3} | 1.053 |

Run #2:

| parameter | value | move_counts | avg_seconds |
|---|---|---|---|
| iterations | 100 | {6: 6, 5: 4, 4: 2, 2: 2, 1: 2, 0: 1, 3: 3} | 0.089 |
| iterations | 500 | {4: 5, 2: 2, 5: 3, 0: 4, 6: 2, 1: 2, 3: 2} | 1.037 |
| iterations | 2000 | {6: 3, 5: 4, 0: 4, 1: 5, 3: 3, 4: 1} | 4.495 |
| expand_x | 2 | {1: 2, 6: 6, 2: 2, 5: 1, 4: 5, 0: 2, 3: 2} | 0.979 |
| expand_x | 4 | {1: 4, 4: 5, 0: 3, 3: 4, 6: 1, 2: 1, 5: 2} | 1.171 |
| expand_x | 7 | {3: 2, 0: 2, 5: 4, 1: 4, 4: 5, 6: 3} | 1.149 |

| | | | |
|---|---|---|---|
| max_depth | 2 | {3: 4, 2: 1, 0: 5, 5: 3, 4: 1, 1: 4, 6: 2} | 0.981 |
| max_depth | 4 | {0: 3, 1: 7, 6: 6, 3: 2, 4: 1, 5: 1} | 1.046 |
| max_depth | 6 | {2: 3, 5: 1, 1: 4, 3: 2, 6: 5, 0: 2, 4: 3} | 1.043 |
| w | 0 | {3: 3, 0: 7, 5: 4, 4: 2, 6: 1, 1: 1, 2: 2} | 1.067 |
| w | 0.5 | {3: 12, 4: 1, 6: 3, 1: 2, 2: 1, 0: 1} | 1.085 |
| w | 1 | {3: 10, 0: 2, 4: 2, 2: 3, 5: 2, 1: 1} | 1.075 |
| c | 0.5 | {2: 5, 3: 4, 5: 4, 0: 4, 4: 1, 6: 1, 1: 1} | 1.042 |
| c | 1.4 | {3: 4, 6: 5, 4: 3, 2: 2, 5: 3, 1: 2, 0: 1} | 1.099 |
| c | 3 | {5: 1, 0: 4, 3: 6, 6: 5, 1: 2, 4: 2} | 1.001 |

Difference per Parameter:

Avg_second:
Iteration has a wide increase in avg_second.
Other parameters are balanced for avg_second.

Across both runs, they present similar results in Avg_second.

Does it produce the same moves?
Yes, parameters in different runs are capable of producing the same moves. For instance:

| | iter100 | iter500 |
|---|---|---|
| run1 | {1: 5, 5: 6, 4: 4, 3: 3, 0: 1, 2: 1} | {0: 3, 3: 5, 2: 5, 4: 3, 5: 2, 1: 2} |
| run2 | {6: 6, 5: 4, 4: 2, 2: 2, 1: 2, 0: 1, 3: 3} | {4: 5, 2: 2, 5: 3, 0: 4, 6: 2, 1: 2, 3: 2} |

Fig 1. Compare move count of iteration 100 and iteration 500 between runs 1 and 2.

Both runs produce 0:1,1:2,3:3 as move counts in both iterations.

Cause of Difference:
Potential it's the length of processing of experimentation as the run #2 ran longer than run#1 and produced greater values but similar range of values in run #1.

## 2. Propositional Logic - Satisfiability (10%)

Decide whether each of the following sentences is valid, unsatisfiable, or neither. Verify your decisions using truth tables or the equivalence rules.

a. Smoke ⇒ Fire

P: Smoke
Q: Fire
P -> Q
P is true when Q is true

| P | Q | P -> Q |
|---|---|---|
| T | T | T |
| T | F | F |
| F | T | T |
| F | F | F |

Therefore, This means that the statement is satifiable as some models are true while others aren't. Therefore, Smoke sometimes implies fires

b. b. ((Smoke ∧ Heat) ⇒ Fire) ⇔ ((Smoke ⇒ Fire) ∨ (Heat ⇒ Fire))

CNF
P : Smoke
Q = Fire
R = Heat

| P | Q | R | PANDR | (PANDR) >Q | P>Q | R>Q | P>Q V R>Q | ((PANDR)>Q) <> ((P>Q) V (R>Q)) | |
|---|---|---|---|---|---|---|---|---|---|
| T | T | T | T | T | T | T | T | T | |
| T | T | F | F | T | T | T | T | T | |
| T | F | T | T | F | F | F | F | T | |
| T | F | F | F | T | F | T | T | T | |
| F | T | T | F | T | T | T | T | T | |
| F | T | F | F | T | T | T | T | T | |
| F | F | T | F | T | T | F | T | T | |
| F | F | F | F | T | T | T | T | T | |

This model is Viable as the statement provided results in model being true in all worlds.

Viable:
True in all
Satisfiable:
True in some
Unsatifiable:
True in No World

## 3. Models (10%)

Consider a vocabulary with only four propositions, A, B, C, and D. How many models are there for the following sentences?

2^4 = 16

a. B∨C.
   1 true
   D and A not here = 2 x 2 x 1 x 1 = 4
   16 – 4 = 12 true models
   4 false models
b. (A ⇒ B) ∧ A ∧ ¬B ∧ C ∧ D
   2^4 = 16 models.
   A true
   A > B, A true means B must be true
   notB forces B to be false.
   Unsatifiable.
   All 16 models are false.
   No True models.

## 4. Proposition Logic - Resolution (20%)

Consider the following sentence: [(Food ⇒ Party) ∨ (Drinks ⇒ Party)] ⇒ [(Food ∧ Drinks) ⇒ Party]
P:Food
Q:Party
R:Drinks

a. Determine, using enumeration, whether this sentence is valid, satisfiable (but not valid), or unsatisfiable.

| P | Q | R | P>Q | R>Q | (P>Q) V (R>Q) | PANDR | (PANDR)>Q | [(P>Q) V (R>Q)] > [(PANDR)>Q] |
|---|---|---|---|---|---|---|---|---|
| T | T | T | T | T | T | T | T | T |
| T | T | F | T | T | T | F | T | T |
| T | F | T | F | F | F | T | F | T |
| T | F | F | F | F | F | F | T | T |
| F | T | T | T | T | T | F | T | T |
| F | T | F | T | T | T | F | T | T |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| F | F | T | T | F | T | F | T | T |
| F | F | F | T | T | T | F | T | T |

This statement is valid as all models display as true.

b. Convert the left-hand and right-hand sides of the main implication into CNF, showing each step, and explain how the results confirm your answer to (a).
P>Q V R>Q > PANDR >Q : Initialize

L: (P>Q) V (R>Q)
R: PANDR >Q

L1 = notPvq v not r v q (remove implifation, delete brackets all are or)

R1 = not(P or R) v q (remove implifation)
R2= notP or notR v q (apply de morgan law, delete brackets all are or)

L and R both become notP or notR or Q, therefore L being true means R is true.

Both method result in the statement being proven True.

c. Prove your answer to (a) using resolution.
not[[(P>Q) V (R>Q)] > [(PANDR) >Q]]]
L: notp v q v notr v q
R: p and r and notq

c1: notp v q v notR v q
c2: p
c3: r
c4: notq
r5: notp v notr or F (1 and 4)
r6: f v not r v F (2,5)
r7: empty (3,6)

By proof of contradiction, negation of original statement being false results in original being true.

# Submission Notes

- Create/Reuse a GitHub repository named: AII501NAA-F2026 on Seneca GitHub Enterprise Cloud
- Within the repository create a folder named Activity2.
- Place all assignment deliverables inside the Activity2 folder, including source code, test cases, test data, documentation, and any supporting files.
- Include a README.md file with setup, execution, testing instructions, and assumptions.
- Add the instructor as a collaborator before the submission deadline.
- Submit the GitHub repository URL through the course submission system.
- Commit history may be reviewed as part of the assessment process.

# Interview Verification and Academic Integrity Policy

To ensure academic integrity and verify authorship of submitted work, you may be selected for an individual interview (viva/oral demonstration).

- Students may be selected randomly or based on the nature of their submission.
- Students must be able to explain their design decisions, implementation details, testing strategy, and use of any AI tools or external resources.
- If a student is selected for an interview, 100% of the assignment grade will be based on the interview performance. The submitted repository will be used only as supporting material during the interview.
- Failure to attend a scheduled interview without an approved accommodation may result in a grade of zero.

Students are responsible for ensuring that they fully understand and can explain all aspects of their submitted work.